

LVM on RHEL

Version 1.0.0, 2026-09-29

Home

Introduction

Course Title: LVM on RHEL

Description: Course description FIXME

Duration: FIXME hours

Topics covered in this course

- Introduction to LVM Components
- Managing Storage with System Roles
- Hands-on Lab

Prerequisites

This course assumes that you have the following prior experience:

- Course or experience...
- Course or experience...
- Course or experience...



Use of Artificial Intelligence: This publication contains content or media that is generated or assisted by AI tools. All the Training deliverables have primarily human authorship, and all content was reviewed by a human domain expert.

Lab Environment

FIXME: Instructions for accessing the lab environment for this hands-on based training.

Introduction to LVM Components

Introduction to LVM Components

Logical Volume Management (LVM) in RHEL provides a flexible approach to storage management by abstracting physical hardware into manageable logical units.

- **Physical Volumes (PVs):** These are the underlying storage devices, such as raw disks or disk partitions, that have been initialized for use by LVM.
- **Volume Groups (VGs):** A Volume Group acts as a storage pool created by combining one or more Physical Volumes. It serves as a container from which logical volumes can be allocated.

Understanding Physical Volumes (PVs)

Understanding Physical Volumes (PVs)

In the Logical Volume Manager (LVM) architecture, the **Physical Volume (PV)** acts as the fundamental building block. Before LVM can manage storage, it must identify and label the underlying hardware to treat it as a flexible pool of data.

What is a Physical Volume?

A Physical Volume is a storage device—or a partition on a storage device—that has been initialized for use by LVM. By converting a raw block device into a PV, you add an LVM metadata header to the disk. This header describes the device as part of the LVM subsystem, allowing the kernel and LVM tools to manage the storage beyond the limitations of standard disk partitions.

Key Characteristics of PVs

- **Abstraction Layer:** PVs hide the physical geometry of the hard drive (or SAN LUN) from the file system.
- **Initialization:** Before a device can be used as a PV, it must be initialized using the `pvcreeate` command. This process writes an LVM label to the start of the device.
- **Flexibility:** Once a device is a PV, it can be assigned to a Volume Group (VG). Because LVM works with the space provided by the PV, you can easily resize, extend, or remove disks from your storage topology without unmounting file systems or repartitioning the hardware.

Physical Volumes and the Storage Stack

In a typical RHEL environment—such as those used for high-performance workloads like SAP HANA—the storage hierarchy is organized as follows:

1. **Physical Device (e.g., `/dev/sdb`):** The raw hardware device.
2. **Physical Volume (PV):** The device after initialization (`pvcreeate /dev/sdb`).
3. **Volume Group (VG):** A collection of one or more PVs acting as a single storage pool.
4. **Logical Volume (LV):** A virtual partition carved out of the VG, formatted with a file system (e.g.,

XFS), and mounted to the directory tree.

Managing PVs in RHEL

While the `rhel_system_roles.storage` role automates these tasks in production environments, understanding the underlying commands is critical for troubleshooting and manual configuration.

Creating a Physical Volume

To initialize a block device for LVM use, execute the following command:

```
[root@localhost ~]# pvcreate /dev/sdb
Physical volume "/dev/sdb" successfully created.
```

Verifying Physical Volumes

To inspect existing PVs on your system and verify their status, metadata, and available capacity, use the `pvs` or `pvdisplay` commands:

```
[root@localhost ~]# pvs
PV          VG   Fmt  Attr PSize   PFree
/dev/sdb    sap  lvm2 a--  <50.00g <10.00g
```

- **PV:** The device name.
- **VG:** The Volume Group to which the PV currently belongs.
- **PSize:** The total size of the physical device available to LVM.
- **PFree:** The amount of unallocated space remaining on that specific PV.

Practical Implications for SAP HANA

In the context of SAP HANA, where performance-critical volumes like `/hana/data` and `/hana/log` are required, LVM allows you to aggregate multiple fast NVMe or SSD drives into a single Volume Group. By creating multiple Physical Volumes from these high-speed devices, you can create large, performant Logical Volumes that meet the specific capacity requirements outlined in your SAP sizing documentation.

Defining Volume Groups (VGs)

Defining Volume Groups (VGs)

In the Logical Volume Manager (LVM) architecture, a **Volume Group (VG)** acts as the central storage pool. It is the abstraction layer that collects one or more Physical Volumes (PVs) and combines them into a single, unified storage entity.

What is a Volume Group?

A Volume Group is the container that sits between the physical storage hardware and the logical volumes (LVs) used by the operating system. By grouping multiple physical disks or partitions together, LVM provides a flexible way to manage space that is no longer restricted by the physical size of an individual disk.

Key characteristics of VGs include:

- * **Storage Pooling:** Multiple PVs of varying sizes can be joined into one VG.
- * **Elasticity:** You can expand a VG at any time by adding new Physical Volumes to it without needing to migrate existing data.
- * **Abstracted Space:** Once the VG is created, space is carved out of the pool to create Logical Volumes.

Creating a Volume Group

To create a Volume Group, you use the `vgcreate` command. This command takes a name for the new VG and a list of Physical Volumes that will contribute their storage to the pool.

```
# Syntax: vgcreate [vg_name] [pv_path1] [pv_path2]...  
[root@localhost ~]# vgcreate vg_data /dev/sdb1 /dev/sdc1
```

In this example, the two partitions, `/dev/sdb1` and `/dev/sdc1`, are initialized as PVs and combined into a single Volume Group named `vg_data`.

Physical Extents (PEs)

When a VG is created, LVM divides the total available space into fixed-size chunks called **Physical Extents (PEs)**.

- * All PEs within a single Volume Group are the same size (the default is usually 4MB).
- * When you create a Logical Volume, LVM allocates these extents to the LV.
- * Because the VG is essentially a collection of PEs, LVM can spread a single Logical Volume across multiple physical disks seamlessly.

Inspecting Volume Groups

To verify the status and capacity of your Volume Groups, use the `vgs` and `vgdisplay` commands.

- **vgs:** Provides a concise, one-line summary of all VGs on the system.

```
[root@localhost ~]# vgs  
VG      #PV #LV #SN Attr   VSize   VFree  
rhel    1   2   0 wz--n- <11.00g 0  
vg_data 2   0   0 wz--n- 19.99g 19.99g
```

- **vgdisplay:** Provides detailed information about a specific VG, including the PE size, the number of free extents, and the physical volumes associated with the group.

```
[root@localhost ~]# vgdisplay vg_data  
--- Volume group ---  
VG Name                vg_data
```

System ID	
Format	lvm2
Metadata Areas	2
VG Size	19.99 GiB
PE Size	4.00 MiB
Total PE	5118
Alloc PE / Size	0 / 0
Free PE / Size	5118 / 19.99 GiB

Extending a Volume Group

One of the primary benefits of using LVM is the ability to grow the VG size online. If you add a new physical disk to your server, you can add it to an existing VG using the **vgextend** command:

```
[root@localhost ~]# vgextend vg_data /dev/sdd1
Volume group "vg_data" successfully extended
```

This immediately increases the **VFree** (Free Space) of the **vg_data** group, allowing you to create or resize Logical Volumes without unmounting or restarting the system.

Managing Storage with System Roles

Managing Storage with System Roles

The `rhel_system_roles.storage` role automates the complex configuration of storage devices on RHEL, ensuring consistency across environments. It simplifies the deployment of logical volumes and file systems by abstracting the underlying storage management tasks.

Key features include:

- **Declarative Configuration:** Defines `storage_pools` and `volumes` in YAML, specifying disks, sizes, mount points, and file system types (e.g., XFS).
- **Automation:** Streamlines the provisioning and formatting of block devices, significantly reducing manual effort and human error.
- **SAP Readiness:** Provides a standardized method to meet specific SAP HANA storage requirements for data, log, and shared directories.
- **Safety Options:** Supports variables like `storage_safe_mode` to manage whether existing storage configurations should be preserved or overwritten during deployment.

Overview of `rhel_system_roles.storage`

Overview of `rhel_system_roles.storage`

The `rhel_system_roles.storage` role is a powerful automation tool designed to abstract the complexities of storage management on Red Hat Enterprise Linux (RHEL). By leveraging Ansible, this role allows system administrators to define complex storage configurations—such as Logical Volume Management (LVM)—as code, ensuring consistency across deployments and reducing the potential for human error.

Purpose and Functionality

In traditional environments, configuring storage often requires executing a sequence of manual commands to initialize physical disks, create volume groups, define logical volumes, and manage file systems. The `rhel_system_roles.storage` role automates this lifecycle by allowing you to define the desired end-state in a YAML-formatted variable file.

The role handles the following operations: * **Initialization:** Preparing raw block devices as Physical Volumes (PVs). * **Abstraction:** Grouping PVs into Volume Groups (VGs). * **Allocation:** Carving out Logical Volumes (LVs) with specific size requirements. * **Lifecycle:** Formatting these volumes with desired file systems (e.g., XFS) and configuring mount points.

Use Case: SAP HANA Deployments

The `rhel_system_roles.storage` role is particularly effective for meeting strict, standardized storage requirements, such as those demanded by SAP HANA. SAP HANA environments require specific sizing for data, logs, and shared volumes based on the system's RAM capacity.

Using this role, you can enforce these standards globally. Instead of manually carving out disks for every new SAP node, you define a single configuration block that ensures every server adheres to your organization's storage policy.

Example Configuration

Below is an example of how the role is configured to provision storage pools and volumes for an SAP HANA environment:

```
storage_pools:
- name: sap
  disks:
    - sdb
    - sdc
    - sdd
    - sde
    - sdf
  volumes:
    - name: data
      size: "128 GiB"
      mount_point: "/hana/data"
      fs_type: xfs
      state: present
    - name: log
      size: "64 GiB"
      mount_point: "/hana/log"
      fs_type: xfs
      state: present
    - name: shared
      size: "128 GiB"
      mount_point: "/hana/shared"
      fs_type: xfs
      state: present
    - name: sap
      size: "50 GiB"
      mount_point: "/usr/sap"
      state: present
```

Key Configuration Considerations

- **storage_safe_mode:** By default, the role provides safety checks to prevent data loss. For fresh installations where you intend to overwrite existing disk configurations, you can set **storage_safe_mode: false**. **Caution:** Use this setting carefully, as it will clear existing storage configurations on the specified devices.
- **Declarative Nature:** Because this role is declarative, you only need to define the "what." The role logic determines the "how," calculating the necessary LVM metadata updates and file system creation commands automatically.
- **Scalability:** Whether managing a single workstation or a large cluster of SAP HANA nodes, this

role ensures that the storage architecture remains identical across the entire infrastructure, simplifying troubleshooting and long-term maintenance.

Configuring storage pools and volumes

Configuring Storage Pools and Volumes

The `rhel_system_roles.storage` role provides a declarative approach to managing LVM storage. Instead of executing manual `pvcreate`, `vgcreate`, and `lvcreate` commands, you define the desired state in an Ansible variable, allowing the system to handle the underlying LVM complexities.

Defining Storage Pools

A **storage pool** acts as the abstraction layer for your physical storage. In the context of `rhel_system_roles.storage`, a pool is essentially a Volume Group (VG) backed by one or more Physical Volumes (PVs).

When you define a pool: 1. **Disks:** You specify the block devices (e.g., `sdb`, `sdc`) that contribute to the pool. The role automatically initializes these as Physical Volumes. 2. **Management:** The role ensures that the Volume Group is created and that the specified disks are included, enabling pooled storage capacity for your logical volumes.

Defining Logical Volumes

Once a pool is established, you define individual **volumes** within that pool. For each volume, you must specify: * **Size:** The capacity required (e.g., `"128 GiB"`). * **Mount Point:** The directory where the volume will be accessible. * **File System Type:** The format (e.g., `xfs` or `ext4`). * **State:** Typically set to `present` to ensure the volume exists.



When using the `rhel_system_roles.storage` role for initial system builds, consider setting `storage_safe_mode: false`. This allows the role to overwrite existing configurations, which is often necessary during fresh provisioning but should be used with extreme caution on systems containing production data.

Example Configuration

The following YAML structure demonstrates how to group multiple disks into a single pool named `sap` and carve out specific volumes for SAP HANA requirements:

```
storage_pools:
  - name: sap
    disks:
      - sdb
      - sdc
      - sdd
      - sde
      - sdf
    volumes:
```

```
- name: data
  size: "128 GiB"
  mount_point: "/hana/data"
  fs_type: xfs
  state: present
- name: log
  size: "64 GiB"
  mount_point: "/hana/log"
  fs_type: xfs
  state: present
- name: shared
  size: "128 GiB"
  mount_point: "/hana/shared"
  fs_type: xfs
  state: present
```

Implementation Considerations

- **Sizing:** Always refer to your application's sizing guide (such as SAP HANA requirements) before defining volumes. For instance, `/hana/log` is typically calculated based on a percentage of system RAM.
- **Automation:** By utilizing this role in an Ansible Playbook, you ensure that storage configuration is idempotent and reproducible across your infrastructure.
- **High Availability:** If your storage configuration involves shared storage for clustering, ensure that the underlying infrastructure is properly configured to prevent "Single Points of Failure," as defined in RHEL High Availability best practices.

Hands-on Lab

Hands-on Lab

The hands-on labs provide a practical environment to apply your knowledge of LVM and storage management. Key activities include:

- **Provisioning:** Creating and formatting block devices to prepare them for data storage.
- **Automation:** Implementing `rhel_system_roles.storage` to configure storage pools and volumes using Ansible roles.
- **Environment Management:** Utilizing the `lab` command-line utility to `start`, `grade`, and `finish` exercises, ensuring a clean and consistent state for each task.

Provisioning and formatting block devices

Provisioning and Formatting Block Devices

In Red Hat Enterprise Linux (RHEL), block device management is the foundation of system storage. Before a raw disk—such as a virtual `vdb` or an unpartitioned physical drive—can be used to store data, it must be properly initialized and prepared for a filesystem.

Understanding Block Device Preparation

Provisioning a new block device typically follows a standard sequence:

1. **Identification:** Locating the raw device (e.g., `/dev/vdb`) using tools like `lsblk`.
2. **Partitioning (Optional):** Creating partitions if specific sub-divisions of the disk are required.
3. **Formatting (Creation of Filesystem):** Placing a filesystem structure (such as XFS or ext4) onto the block device or partition so the OS can organize files.
4. **Mounting:** Attaching the formatted device to the directory tree.



Before performing any operations on a disk, always verify the device name using `lsblk` to avoid data loss on existing system drives.

Hands-on Lab: Provisioning and Formatting

In this exercise, we will take a raw 50GB disk (`/dev/vdb`) and prepare it for use.

Step 1: Identify the Disk

First, verify the presence of the unformatted disk. As seen in the system context, your environment may show a raw device without a mount point:

```
[student@localhost ~]$ lsblk
```

```
NAME    MAJ:MIN RM   SIZE RO TYPE MOUNTPOINT
vda     252:0    0   200G  0 disk
├─vda1  252:1    0     1M  0 part
├─vda2  252:2    0   100M  0 part /boot/efi
└─vda3  252:3    0  199,9G  0 part /
vdb     252:16   0     50G  0 disk
```

Step 2: Create a Filesystem

For most modern RHEL deployments, XFS is the recommended filesystem due to its performance and scalability. We can format the raw block device directly without creating a partition table if the entire disk will be used as a single volume.

Run the following command to format `/dev/vdb` with the XFS filesystem:

```
[root@localhost ~]# mkfs.xfs /dev/vdb
```

Step 3: Verify the New Filesystem

Once the formatting process completes, verify that the device is ready by checking the file system type:

```
[root@localhost ~]# blkid /dev/vdb
/dev/vdb: UUID="a1b2c3d4-e5f6-7890-abcd-1234567890ab" BLOCK_SIZE="4096" TYPE="xfs"
```

Step 4: Mount the Device

To use the storage, create a mount point and attach the device:

```
[root@localhost ~]# mkdir -p /mnt/data
[root@localhost ~]# mount /dev/vdb /mnt/data
[root@localhost ~]# df -h /mnt/data
Filesystem      Size  Used Avail Use% Mounted on
/dev/vdb         50G   400M   50G   1% /mnt/data
```

Troubleshooting Guidance

- **Device Busy:** If you cannot format a device, ensure no processes are accessing it and that it is not currently mounted (`umount /dev/vdb`).
- **Persistent Mounts:** The `mount` command is temporary. To ensure the disk mounts automatically after a reboot, add the UUID identified in Step 3 to the `/etc/fstab` file:

```
UUID=a1b2c3d4-e5f6-7890-abcd-1234567890ab /mnt/data xfs defaults 0 0
```

Automating storage configuration using Ansible roles

Automating Storage Configuration with Ansible

In enterprise environments, manually configuring Logical Volume Manager (LVM) components across multiple nodes is error-prone and inefficient. The `rhel-system-roles` collection, specifically the `storage` role, provides a standardized, idempotent way to manage physical volumes, volume groups, and logical volumes.

Overview of `rhel_system_roles.storage`

The `rhel_system_roles.storage` role simplifies the complexity of block device management by allowing you to define your desired storage state in YAML variables rather than writing individual shell commands.

By using this role, you can: * **Provision** physical devices into Volume Groups (VGs). * **Create** Logical Volumes (LVs) with specific sizing and file system types. * **Mount** volumes automatically by defining mount points and options.

Core Concepts

- **Disks:** The raw physical block devices.
- **Volume Groups:** A pool of storage created from one or more physical volumes.
- **Logical Volumes:** The usable storage partitions created from the Volume Group space.

Hands-on Lab: Automating Storage with Ansible

This lab demonstrates how to configure an LVM setup using an Ansible playbook.

Prerequisites

Ensure you have the `rhel-system-roles` collection installed on your control node:

```
ansible-galaxy collection install redhat.rhel_system_roles
```

Step 1: Define the Storage Variables

Create a file named `storage_vars.yml`. This file defines the desired state of your storage.

```
storage_volumes:
  - name: my_vg
    type: lvm
    disks: ["/dev/sdb", "/dev/sdc"]
    volumes:
      - name: data_lv
        size: 10g
        mount_point: /mnt/data
        fs_type: xfs
```

```
state: present
```

Step 2: Create the Playbook

Create a playbook named `configure_storage.yml` to apply the role.

```
---
- name: Automate LVM configuration
  hosts: managed_nodes
  vars_files:
    - storage_vars.yml
  roles:
    - redhat.rhel_system_roles.storage
```

Step 3: Execute the Playbook

Run the playbook to provision the storage. The role will identify the disks, create the Volume Group (`my_vg`), format the Logical Volume (`data_lv`), and handle the mounting process.

```
ansible-playbook -i inventory configure_storage.yml
```

Verification

After the playbook completes, verify the new LVM structure on your managed node:

```
# Check the Volume Group
vgs

# Check the Logical Volume
lvs

# Check the mount point
df -h | grep /mnt/data
```

Troubleshooting Tips

- **Device Identification:** Ensure the device names (e.g., `/dev/sdb`) are consistent across your environment. Using UUIDs or labels is recommended for production to avoid issues if device paths change.
- **Idempotency:** If the role fails, check if the disks are already in use or part of a different volume group. The `rhel_system_roles.storage` role will report any conflicts that prevent the desired state from being reached.
- **Dependencies:** Ensure that `lvm2` packages are available in your repositories, as the role relies on these binaries being present on the target system.

Appendix

FIXME: Content for Appendix...

Resources

[Download Course PDF](#)